

OBJECT-ORIENTED PROGRAMMING (OOP)

1. What is OOP?

Object-Oriented Programming, or OOP, is a way of writing programs using **classes and objects**. In OOP, related data and functions are kept together. An object represents a particular thing or entity and contains the information and functions related to it.

The main aim of OOP is to make programs easier to understand, develop and maintain. A big program can be divided into smaller parts, so it becomes easier to work with and reuse them. C++, Java and Python are some languages that support OOP.

2. Procedural Programming

Procedural programming is a programming method where a program is divided into different **functions or procedures**. Each function is made to perform a particular task. The main focus is on the steps that are followed to solve a problem.

In this type of programming, data and functions are usually kept separate. When the program becomes large, it can become difficult to manage many functions and data. C is a common example of a procedural programming language.

3. Object-Oriented Programming

Object-Oriented Programming mainly works with **objects**. An object contains data and the functions that work on that data. Objects are created using classes, which tell us what data and functions the objects will have.

OOP helps us divide a large program into smaller and understandable parts. It also provides concepts like **encapsulation, inheritance, abstraction and polymorphism**. These concepts help in making programs organized and reusable.

4. Difference Between Procedural Programming and OOP

Basis	Procedural Programming	Object-Oriented Programming
Main focus	Mainly focuses on functions and procedures.	Mainly focuses on objects and classes.
Program design	Usually follows a top-down approach.	Usually follows a bottom-up approach.
Data and functions	They are generally kept separate.	Related data and functions are kept together.
Data security	There is less control over direct access to data.	Encapsulation gives better control over data access.
Code reuse	Code reuse is possible but more limited.	Code can be reused using classes and inheritance.
Large programs	Can become difficult to manage when the program grows.	Can be divided into smaller and easier-to-manage parts.
Maintenance	A change in one part may affect other parts.	Changes can often be limited to a particular class or object.
Examples	C, Pascal and similar languages.	C++, Java, C# and Python.

5. Basic Concepts of OOP

Objects

An object is one of the basic parts of OOP. It represents a particular thing and contains its related data and functions. The data tells us the current state of the object, while the functions tell us what actions the object can perform.

Objects are made from classes. We can create many objects from the same class, but each object can have

different values. Objects can also communicate with each other using functions.

Class

A class is like a **blueprint or template** for creating objects. It tells us what data and functions an object will have. It keeps related information and operations together.

A class only describes the structure, while an object is the actual instance of that class. A class can have data members to store information and member functions to perform operations.

Entity

An entity is any identifiable thing about which we can store or process information. It can be a person, place, physical thing or even an idea.

In OOP, an entity can be represented using a class, and its individual instances can be represented using objects. Finding the entities in a problem helps us decide which classes and objects are needed in the program.

Data Abstraction

Data abstraction means **showing only the important information and hiding unnecessary details**. The user mainly needs to know what an operation does and does not need to know all the internal steps used to perform it.

Abstraction makes a program easier to understand and reduces complexity. The internal working of a class can also be changed without changing the way other parts of the program use it.

6. Types / Main Concepts of OOP

Inheritance

Inheritance is an OOP concept in which a new class gets properties and functions from an existing class. The existing class is called the **base or parent class**, and the new class is called the **derived or child class**.

Inheritance is mainly useful for code reuse. Instead of writing the same properties and functions again, the new class can use the existing ones and add its own features. It also creates a relationship between related classes.

Polymorphism

Polymorphism means **more than one form**. In OOP, it allows the same function, operation or interface to work differently in different situations. The behavior can depend on the object or on how the function is used.

It makes programs more flexible because the same type of operation can be used with different objects while giving suitable results for each object.

Abstraction

Abstraction means hiding the unnecessary implementation details and showing only the important information. It allows the user or programmer to focus on the required operation instead of worrying about every internal detail.

It reduces the complexity of a program and makes classes easier to use. The user mainly needs to know what operations are available, not exactly how they work internally.

Encapsulation

Encapsulation means putting **data and the functions that work on that data together in one unit**, usually a class. It keeps related information together and controls how that information can be accessed.

Encapsulation also helps in data hiding. In C++, access specifiers such as **private, protected and public** are used to control access to data. This helps prevent unwanted changes and keeps the program organized and secure.

Conclusion

OOP is a useful way of organizing programs using classes and objects. Its basic concepts include objects, classes, entities and abstraction. Its important principles include inheritance, polymorphism, abstraction and encapsulation. Understanding these concepts gives a good base for learning C++ and other advanced OOP topics.